18. Distributed Job Scheduler / Task Queue

> **Interviewer:** Design a distributed job scheduler — think of it as the engine behind "send this email in 5 minutes," "run this report every day at 2am," or "process this video right now." Users submit jobs; the system must run each one reliably, on some worker, even if machines crash. Where do you start?

Candidate: This is fundamentally a **"make sure work happens, exactly when it should, exactly (or almost exactly) once" problem**, not a read-scaling problem. Let me clarify the shape before I design.

---

1. Clarifying the requirements

> **Candidate:** A few questions: 1. Three kinds of jobs, right — **run now**, **run at a specific future time** (delayed), and **recurring** (cron-style, "every day at 2am")? 2. If a job fails — say the downstream API is down — do we retry automatically, or is that the job's own responsibility? 3. If a job accidentally runs **twice**, is that catastrophic (e.g., double-charging a card) or just wasteful (e.g., re-sending one email)? 4. What scale are we talking — thousands of jobs a day, or millions?
>
> **Interviewer:** All three job types. Automatic retries with backoff, then give up and flag it for a human. Assume some jobs are money-sensitive — duplicate execution must be *made safe*, not just "rare." Scale: tens of millions of jobs a day, many recurring.

Candidate: Good, that pins down the hard part. Requirements:

Functional - `POST /jobs` — submit a job: immediate, delayed (`run_at`), or recurring (`cron`). - `GET /jobs/{id}` — status: `pending | claimed | running | succeeded | failed | dead_letter` . - `DELETE /jobs/{id}` — cancel a not-yet-run job. - Automatic **retry with backoff**; after `max_attempts` , move to a **dead-letter queue (DLQ)**.

Non-functional — this is where it's interesting - **Never lose a job.** Worst case it runs late or twice — **never zero times.** - **No duplicate execution for money-sensitive jobs** — needs idempotency, because true exactly-once delivery across a crash-prone distributed system is not achievable for free. - **Timeliness within seconds**, not real-time — reliability beats raw latency. - **Scale:** tens of millions of jobs/day, thousands of distinct recurring schedules, thousands of workers pulling concurrently.

---

2. Estimation — sizing the "due job" problem

> **Candidate:** Let me size this so I know whether one database can hold the job table and how many workers I need.

```
30M jobs/day submitted.
  average = 30,000,000 / 86,400 ≈ 350 jobs/sec
  peak (many daily crons fire at 2am / on the hour together) ≈ 5–10x average
    ≈ 2,000–3,500 jobs/sec at peak minute

Job row ≈ 1 KB (type, payload ref, schedule, status, attempts).
  30M/day * 1KB ≈ 30 GB/day of metadata. Keep 30 days "hot" ≈ ~1 TB —
  comfortably a single sharded Postgres/MySQL cluster; archive older rows.

Worker capacity: avg job takes ~2s of runtime.
  3,000 jobs/sec (peak) * 2s ≈ 6,000 concurrent job-slots needed
  -> a few hundred to low-thousands of worker machines, horizontally scaled.

=> The core mechanical challenge is NOT storage size — it's:
   "how do we find the jobs that are due RIGHT NOW, out of tens of millions
   of rows, fast enough, without two schedulers grabbing the same job?"
```

---

3. The naive V1 — poll a table, and why it's fragile

> **Candidate:** Let me state the simplest version, because its failure modes drive the design.

```
  [ Client ] --POST /jobs--> [ API ] --> [ jobs table (Postgres) ]

  [ Scheduler, single process ]  every 1s:
       SELECT * FROM jobs WHERE status='pending' AND run_at <= now()
       for each row: mark 'running', execute it INLINE, mark 'succeeded'/'failed'
```

Why this dies at scale: a single scheduler process is a throughput ceiling *and* a single point of failure — it can't run 3,000 jobs/sec inline. A full-table scan every second to find "due" rows melts the DB as the table grows past tens of millions of rows. Naively adding a second scheduler for HA means **two schedulers can grab the same due row** — double execution. And a crash mid-execution leaves a job stuck in `running` forever, with nobody retrying it.

So V1 tells me the real requirements: **(a)** decouple *deciding a job is due* from *running it*, **(b)** make the "find due jobs" query cheap and indexed, **(c)** make claiming a job atomic, and **(d)** detect stuck jobs via a lease, not a crash-proof promise.

---

4. V1 that actually works — index + atomic claim + pull workers

Candidate: Three pieces, and I'll draw the pipeline first.

```
[ Job DB (Postgres) ]  <-- every job + schedule + status, indexed by (status, run_at)
        |
        |  Scheduler polls: "WHERE status='pending' AND run_at <= now() LIMIT 500"
        v
[ Scheduler(s) ]  atomically flips due rows: pending -> claimed, pushes onto...
        |
        v
[ Queue: Kafka / SQS ]   <-- the "ready to run RIGHT NOW" list
        |
        v
[ Worker pool ]  PULLS jobs, runs them, reports status back to Job DB
```

- **Index `(status, run_at)`** turns "find due jobs" from a full scan into a cheap range lookup — this is the load-bearing index of the whole system.
- **Atomic claim**, one UPDATE, is how we avoid two schedulers or two workers touching the same job: `sql UPDATE jobs SET status='claimed', locked_by=$me, locked_at=now() WHERE id = $id AND status='pending' -- only ONE caller's UPDATE can flip 0 rows -> row_count rows; the loser gets 0 rows`
- **Workers pull, they are never pushed to.** A worker only asks for more work when it has a free slot — that's natural back-pressure, and it means there's nothing to load-balance *toward* a worker; the queue does the spreading.

---

5. The due-time problem, precisely — time-wheel vs. naive polling

> **Interviewer:** Polling `WHERE run_at <= now()` every second across millions of rows — even with the index — still means constant DB traffic. And cron jobs mean *recomputing* "when's next" all the time. How do you make triggering-at-the-right-time cheap?

Candidate: Two complementary moves.

1. Precompute `next_run_at`, don't parse cron on every tick. Each recurring schedule is one row; a background job computes its *next* fire time once and stores it. The scheduler only ever asks "what's due before now," never "what does this cron string mean right now."

2. A time-wheel / bucketed due-time index in front of the DB, so the scheduler isn't hitting Postgres every second for the *global* answer:

```
Conceptual "time wheel" — buckets keyed by second/minute, each bucket
holding the job IDs (or a pointer) due in that slot:

 now=12:00:00           12:00:01         12:00:02   ...  12:00:59
┌─────────────┐      ┌─────────────┐   ┌─────────────┐   ┌─────────────┐
│ job 7,19    │ ──tick──▶│ job 3       │ ─▶│ (empty)     │ .. │ job 42      │
└─────────────┘      └─────────────┘   └─────────────┘   └─────────────┘
 The scheduler advances one bucket per second, dispatches whatever is
 in the CURRENT bucket, and never scans the whole table to do it.
```

In-memory (Redis-backed) time wheels are the classic approach for **near-term** due jobs (the next few minutes) — cheap `O(1)`-ish dispatch per tick. Jobs due **far in the future** (a report scheduled next month) stay in the DB; a slower sweep (every few minutes) promotes "coming up soon" jobs into the near-term

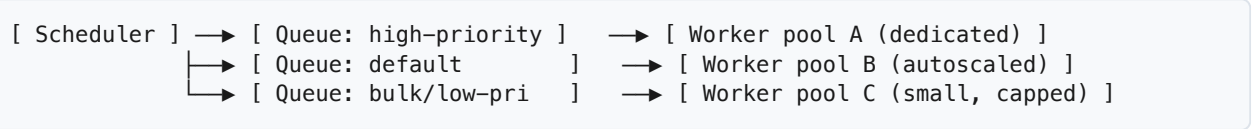
wheel — coarse storage for the far future, a fine-grained hot structure for "about to fire," the same two-tier trick as a calendar reminder app. **Why not just poll Postgres forever?** Fine at moderate scale (say this tradeoff aloud), but at thousands of due-jobs/sec, repeated range scans plus lock contention on the same hot time window becomes the bottleneck; the wheel absorbs that churn in memory instead.

---

6. Scaling workers — 10M jobs/day and hot job types

> **Interviewer:** Now say one job type — "send push notification" — gets submitted a million times in a burst (a marketing blast). What happens to everyone else's jobs?

Candidate: Without isolation, that flood **starves every other job type**, since they all compete for the same worker pool and queue. Fix: **per-type (or per-priority) queues**, with workers subscribed according to capacity plans.



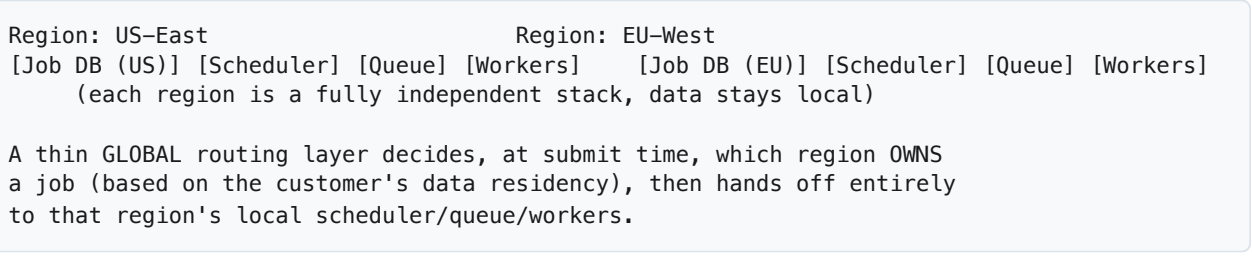
Priorities become "which queue," not a field workers interpret — pool B never sees bulk-queue backlog, so a blast can't delay a password-reset email. Autoscale on queue depth ("if `default` depth grows, add workers to pool B") — the worker layer is stateless, easy to scale out. GPU / capability-tagged jobs work the same way: a dedicated queue + a worker pool that only pulls jobs matching that capability.

---

7. Multi-region

> **Interviewer:** Now this runs globally — US and EU customers. How do you split it?

Candidate: Jobs are **pinned to a region**, not globally replicated — unlike, say, an immutable URL mapping. A job's payload can contain customer PII, and running a EU customer's job on a US worker can be a **GDPR violation**, not just a latency hit.



Global coordination for *placement*, regional isolation for *data and execution*. If a job truly must run "anywhere" (no data sensitivity), the routing layer can pick the nearest region for latency; that's the exception, not the default.

---

8. Latency — where the seconds go, and cutting them

> **Interviewer:** A user submits a "run in 5 seconds" job. Where does time actually go before it executes, and where can you shave it?

Candidate:

```
submit -> DB write (ack to user)         ~10–20ms   (this part IS synchronous)
DB write -> scheduler notices it's due    up to 1 tick of the polling/wheel interval
scheduler claims -> pushes to queue       ~5–10ms
queue -> worker pulls it                  depends on worker poll interval / push notify
```

The biggest lever is **time-wheel granularity** — a 1-second wheel means up to ~1s of avoidable delay; tightening to sub-second buckets trades scheduler CPU for latency, worth it for "seconds" SLAs, not for "run at 2am." **Workers should long-poll the queue**, not poll every few seconds, to remove the "queue has it, nobody's asking" gap. And nothing synchronous or slow belongs in the claim path — the scheduler only flips status + enqueues; actual execution (calling external APIs, etc.) always happens on a worker, off the scheduler entirely.

---

9. Consistency — the exactly-once conversation

> **Interviewer:** Here's the one I really want to dig into: a worker finishes charging a customer's card, but crashes **before** it can tell the Job DB "succeeded." What happens?

Candidate: This is the crux of the whole problem. The honest answer: **true exactly-once execution is not achievable for free in a distributed system** — you'd need an atomic, crash-proof handoff between "the side effect happened out in the world" and "we recorded that it happened," and those are two different systems that can't commit as one transaction.

So we commit to at-least-once execution, and make duplicates harmless instead of impossible:

```
THE FAILURE:
  worker claims job (lease, TTL 5 min) -> charges card -> CRASHES before
  reporting success -> lease expires -> another worker claims the SAME
  job -> charges the card AGAIN.

THE FIX — idempotency, not perfection:
  Every job carries an idempotency_key describing the INTENT, not the
  attempt: "charge order #123" (not "run job #4821").
  Before doing the side effect, the job's own logic checks a dedup store:
    "have I already completed order #123's charge?"
      yes -> skip, return the cached result
      no  -> do the charge, atomically record the key as done
  Design payloads to be idempotent BY CONSTRUCTION where possible:
    "set invoice status = PAID"  (idempotent)
    NOT "increment paid_count by 1"  (not idempotent)
```

Strong / linearizable: the `pending -> claimed` flip (one atomic UPDATE/CAS) and the idempotency-key check-and-set — these two atomic operations prevent double-charging even though the *job* may run

twice. **Eventual:** the job's `status` as seen by a polling dashboard — it might show `running` for a few seconds after it actually finished, harmless. **The pairing to say out loud:** "*At-least-once delivery, plus idempotent jobs, gives the same user-visible guarantee as exactly-once, at a fraction of the complexity.*"

---

10. Async — the queue and workers ARE the core of this design

> **Interviewer:** Where's the async boundary here — what's on the critical path vs. not?

Candidate: Unusually for this kind of interview question, **the entire system to the right of "submit" is async** — that's the whole point of a job scheduler.

```
ON the critical path (user waits for this):
  POST /jobs -> validate -> write row to Job DB -> return job_id
  (a few ms — a synchronous DB write, nothing more)

OFF the critical path (everything else):
  Job DB -> Scheduler (polls/wheel) -> Queue -> Worker -> execute
         -> report status back to Job DB -> (optional) webhook/notification
```

The user gets a `job_id` immediately and either **polls** `GET /jobs/{id}` or (better, for high-value jobs) registers a **webhook** so we push a callback on completion instead of making them poll. **The queue is the shock absorber:** if workers fall behind (a slow downstream API), jobs pile up in the queue rather than backing up into the Job DB or blocking new submissions — submitters never feel worker-side slowness.

---

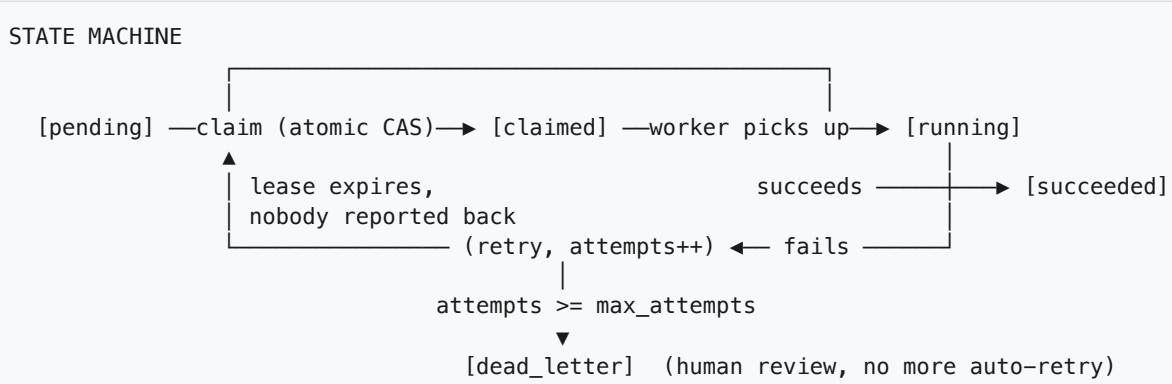
11. Caching

> **Interviewer:** Is there much to cache here?

Candidate: Less than in a read-heavy system — this isn't a URL shortener — but two small wins. **Status polling:** if a dashboard polls `GET /jobs/{id}` every second, cache `(job_id -> status)` in Redis with a short TTL — DB stays the source of truth, cache absorbs repeat reads. **"Next due" lookups:** the scheduler keeps an in-memory min-heap / time-wheel (section 5) of upcoming due times per shard, so it only re-queries the DB when something is imminent — same idea as a cache: don't re-derive a value that hasn't changed. Data flow: `Job DB -> Redis (write-through on status change) -> client poll (read from cache)`. No delicate invalidation needed — statuses only move forward through the state machine, so a stale cached status is at worst a few seconds behind, never wrong in a way that matters.

---

12. Job state machine and the claim/lease flow




```
DISPATCH / LEASE FLOW (the concurrency-safety core)
  Scheduler:  UPDATE jobs SET status='claimed', locked_by=SCHED_1, locked_at=now()
                WHERE id=42 AND status='pending'
              -> 1 row updated? push job 42 onto queue. 0 rows? someone beat us, move on.

  Worker:     pull job 42 off queue
              UPDATE jobs SET status='running', locked_by=WORKER_7, lease_until=now()+5m
                WHERE id=42 AND status='claimed'
              -> execute -> on success: UPDATE status='succeeded'
                         -> on failure: attempts+=1; if attempts<max: status='pending',
                            next_retry_at = now() + backoff(attempts)  (1s, 2s, 4s, 8s...)
                            else: status='dead_letter'

  Lease sweep (runs continuously):
  UPDATE jobs SET status='pending' WHERE status='running' AND lease_until < now()
              -> a crashed worker's job silently becomes claimable again.
```

Why a lease and not a permanent lock? A lock a crashed worker holds forever is a job that never runs again. The lease (`locked_at` / `lease_until`) auto-expires, so a sweeper can put the job back into circulation — the tradeoff is exactly the at-least-once guarantee from section 9: the first worker might have half-finished the work before dying, hence idempotency is mandatory, not optional.

13. Technology choices — what and why (the detailed part)

Concern	Choice	Why this, and why not the alternative
Job metadata + schedule store	**PostgreSQL/MySQL** (`jobs` table, index on `(status, run_at)`)	Claiming a due job is a transactional "atomically flip pending→claimed" operation — exactly what a relational DB's row-level locking / CAS UPDATE guarantees. *Not a plain NoSQL KV store* — single-row atomic updates work fine there too, but "find all jobs due before now" is a range query that SQL indexes handle naturally; a KV store would need a hand-rolled secondary index to do the same job.
Near-term due-job dispatch	**In-memory time-wheel (Redis-backed) for the next few minutes**	O(1)-ish bucketed dispatch avoids repeated full-table range scans on a hot, constantly-advancing time window. *Not pure DB polling for everything* — it works fine at low volume (say this tradeoff aloud) but at thousands of due-jobs/sec, hammering Postgres every second for the global "what's due" answer becomes the bottleneck.

Concern	Choice	Why this, and why not the alternative
"Ready to run now" hand-off	**Message queue (Kafka or SQS)** between scheduler and workers	Durable, replicated, built for many consumers pulling concurrently — far cheaper than workers polling the DB directly at high frequency. *Not skipping the queue* — fine as a simplification at small scale, but at real scale it re-introduces DB hot-spotting on the jobs table.
Preventing double execution	**Atomic CAS UPDATE (`WHERE status='pending'`) for claiming + idempotency key for the job's side effect**	The CAS makes claiming exactly-once *among schedulers/workers*; the idempotency key makes the job's *external effect* safe even if it runs twice after a crash. *Not a distributed lock service (e.g., a separate Redlock/ZooKeeper lock per job)* — that adds an extra moving part and failure mode for something a single atomic DB UPDATE already guarantees per row.
Cross-process mutual exclusion for the scheduler itself	**Leader election (etcd/ZooKeeper)** so exactly one scheduler is "active," with instant standby failover	Avoids two full scheduler processes both scanning/dispatching and doubling work; a dead leader must not mean "no jobs get picked up for hours." *Not "just run N schedulers and let the DB sort it out"* — works for the *claim* (CAS handles that), but without leader election you still get wasted duplicate polling and messier failure diagnosis at scale.
Why not just cron on a single box?	Rejected outright for anything beyond a toy	Plain cron has no retries, no distributed claim, no visibility into failures, and is a single point of failure — the exact opposite of what "reliably run jobs at scale" requires. It's the strawman that motivates this whole design.
Dead-letter handling	**Separate DLQ / `dead_letter` status**, human-reviewed	Retrying forever wastes worker capacity on a job that will never succeed (a "poison" job); a DLQ isolates it for a human instead of silently dropping it or looping it forever.
Priorities / hot job types	**Per-priority (or per-type) queues + dedicated worker pools**	Isolates a flooding low-priority job type from starving urgent ones; a worker pool never even sees backlog it's not subscribed to. *Not one global queue with a priority field* — a single congested queue still makes low-priority jobs physically block the front of the line under most queue implementations.

The one-sentence justification you'd say out loud: "*A relational store gives me the atomic claim that prevents double-dispatch and an indexed range query for 'what's due'; a time-wheel absorbs the constant churn of near-term due-time checks so I'm not hammering that table every second; a durable message queue decouples 'decided it's due' from 'actually ran it' so workers just pull at their own pace; and because true exactly-once is unaffordable in a crash-prone distributed system, I commit to at-least-once delivery made safe by idempotency keys, backed by leases so a crashed worker's job always comes back to life instead of vanishing.*"

---

14. Failure modes & graceful degradation

Failure	What happens	Mitigation
Worker crashes mid-job	Job stuck in `running`	Lease (`lease_until`) expires; sweeper flips it back to `pending`; another worker retries — idempotency makes the retry safe
Job repeatedly fails	Could hammer a broken dependency forever	Exponential backoff (1s, 2s, 4s, 8s...) between retries; after `max_attempts`, move to DLQ for human review
Scheduler (leader) dies	No new jobs get dispatched	Standby scheduler promoted via leader election within seconds; already-queued jobs keep running unaffected
Queue broker dies	A "ready now" job could be lost	Queue's own replication (Kafka/SQS) means the message survives; consumers reconnect to a surviving broker
Job DB primary dies	Can't accept new submissions / claims briefly	Promote a replica; jobs already sitting in the queue continue running — they don't depend on the DB primary being up
Thundering herd (thousands of "daily 2am" jobs fire at once)	Worker pool spikes instantly	**Jitter**: add a small random offset to each schedule's fire time so they spread over, say, a 60s window instead of one instant
"Poison" job (always crashes the worker process itself)	Could repeatedly take down workers	Lease expires, retried a bounded number of times, then DLQ — never retried forever

15. Follow-up curveballs

> **Interviewer:** How do you support cron efficiently when you have thousands of schedules?

Candidate: Never parse the cron string on every scheduler tick. Precompute each schedule's `next_run_at` once (when it's created and again each time it fires), store it as a normal timestamp, and let the same `(status, run_at)` index / time-wheel handle it identically to a one-off delayed job. Cron parsing becomes a rare, cheap operation instead of a hot-path one.

> **Interviewer:** A job needs a GPU. How do you make sure it lands on the right worker?

Candidate: Tag jobs with required capabilities (`gpu: true`) at submit time, and route them to a dedicated queue that only GPU-equipped workers subscribe to — the same per-type-queue mechanism used for priorities in section 6, just keyed on capability instead of urgency.

> **Interviewer:** Marketing wants to see "jobs completed today" as a live dashboard number.

Candidate: Never compute that by scanning the hot `jobs` table live. Emit a lightweight completion event to a metrics pipeline (or increment a Redis counter) whenever a job succeeds, and let the dashboard read that aggregate — same principle as the "burgers remaining" counter in an earlier problem: keep cosmetic/analytics reads far away from the transactional path.

> **Interviewer:** Two workers both think they hold the lease on the same job because of a clock skew issue. What now?

Candidate: Don't trust wall-clock comparisons alone across machines for something this sensitive — use the DB's own atomic CAS as the tiebreaker, not "my local clock says my lease is still valid." Concretely: the `running -> succeeded` transition itself is a CAS (`WHERE status='running' AND locked_by=$me`), so even if a second worker's sweep prematurely decided the lease expired and grabbed the job, only one of the two competing "mark succeeded" writes will actually succeed against the row; the loser's write affects 0 rows and it discards its result. Combined with an idempotent side effect, this is safe even in the ugly case.

---

16. Deep-dive: "Why not just plain cron, or a simple DB-polling loop?"

> **Interviewer:** All this machinery — atomic claims, leases, queues, time-wheels. Why not just put a **cron job on a box** that fires a script, or a **loop that polls the jobs table** every second and runs whatever's due, inline, right there? It's a lot less infrastructure.

Candidate: It's the right instinct to ask — for a handful of internal jobs, plain cron on one box is genuinely the correct answer, and I'd say that out loud rather than over-engineer. But the requirements we pinned down in section 1 — tens of millions of jobs/day, automatic retries, money-sensitive dedup, visibility into failures — break both "simple" options in specific, predictable ways.

Why single-box cron breaks first

```
[ Box A: cron ] --00:00,00:05,...--> runs script inline, on THAT box only

Box A's disk dies at 2:03am.
     |
     ▼
Cron is gone. No other process knows those jobs existed. No retry,
no alert beyond "the box is down," no record of what was scheduled,
no record of what almost ran. Every job silently stops forever.
```

Cron's job list lives **only in that box's crontab**, and its execution history lives only in that box's stdout/log file (if anyone's watching). There is no external durable state — so the box **is** the job store, the scheduler, and the worker, all three single points of failure fused into one. A failure anywhere is total.

Why a naive DB-polling loop breaks second, differently

A polling loop (section 3's V1) fixes "durable state" — jobs live in a table, not a crontab — but if you try to make it *highly available* the naive way (just run two copies of the loop for redundancy), you re-introduce the exact problem HA was supposed to solve:

```
Poller A: SELECT * FROM jobs WHERE run_at <= now() AND status='pending'
Poller B: SELECT * FROM jobs WHERE run_at <= now() AND status='pending'
                       (both run within the same 1s tick)

Both SELECTs return job #42 (nobody has claimed it yet)
     │                                │
     ▼                                ▼
Poller A executes job 42       Poller B ALSO executes job 42
     │                                │
     └──────────── double execution, same instant ────────────┘
```

This is the classic **thundering herd of schedulers**: adding boxes for availability multiplies the chance of double-firing instead of preventing it, because "read then act" is not atomic — two readers can both see "pending" before either one writes "claimed." (This is precisely why section 4 makes the claim a single atomic `UPDATE ... WHERE status='pending'` instead of a `SELECT` followed by a separate `UPDATE`.) And even with one poller, inline execution means a slow job (a hung HTTP call) blocks the *next* tick's poll — the scheduler and the worker being the same process is a latency and blast-radius problem on top of the HA problem.

Comparison

Concern	Single-box cron	Naive DB-polling loop (1+ copies)	Distributed scheduler (durable store + queue + workers)
Single point of failure	Total — box dies, every job silently stops, no record it existed	Partial — job state survives in the DB, but a lone poller dying still halts dispatch until it's restarted	None — leader election fails over the scheduler role; workers are stateless and horizontally replaceable
Double-firing under redundancy	N/A (no redundancy to speak of)	**Yes** — two pollers can both see the same "pending" row before either claims it; adding boxes makes it worse, not better	No — atomic CAS claim (`WHERE status='pending'`) guarantees only one caller's UPDATE affects the row
Retries	None — script fails, nobody notices unless someone greps a log	Bolt-on at best — retry logic has to be hand-rolled into the same fragile loop	Built in — `attempts`, exponential backoff, DLQ after `max_attempts` (sections 1, 12, 14)
Visibility / status	None — no `GET /jobs/{id}`, no history, no dashboard	Whatever you build yourself on top of the table	First-class — state machine (`pending/claimed/running/succeeded/dead_letter`), queryable by design

Concern	Single-box cron	Naive DB-polling loop (1+ copies)	Distributed scheduler (durable store + queue + workers)
Throughput at scale	One box, one script at a time	Caps out fast — inline execution on the poller serializes work; scanning the whole table every tick doesn't scale past low volume	Scales horizontally — scheduler only flips status + enqueues; workers pull and execute off the critical path (sections 4, 10)
Where it's actually the right call	A handful of internal, non-critical jobs, one team, failure is a shrug	A prototype, or genuinely low volume with tolerance for occasional missed/duplicate runs	Anything with real reliability, retry, dedup, or scale requirements — i.e., this problem

> **Rule of thumb:** *Cron is a scheduler with no memory and no witness — fine until you need a second box or a retry.* A polling loop gives you memory (the DB) but not safety under redundancy unless the claim itself is atomic. The distributed design isn't "more infrastructure for its own sake" — durable store, atomic claim, and a queue are the minimum set of pieces that make retries, dedup, and horizontal scaling *possible at all*.

---

17. Deep-dive: the lease-expires-mid-execution double-run

> **Interviewer:** Walk me through the sharpest version of the double-execution problem. Not "the worker crashed" — the worker is *alive*, it's just slow. What happens?

Candidate: This is the nastiest variant because nothing actually failed — there's no crash to catch, no error to log. A worker just pauses for longer than its lease, and by the time it wakes back up, the system has already decided it's dead and handed the job to someone else.

Why it happens

The lease (section 12) is a bet: "if I don't hear back within N minutes, assume the worker is gone and let someone else try." A **GC pause, a blocked network call, a CPU-starved container, a swapped-out VM** — any of these can stall a worker process for longer than the lease TTL *without killing it*. The worker isn't dead; it's just not making progress the sweep can see.

The timeline, in detail

```
t=0s     Worker-A: UPDATE jobs SET status='running', locked_by='A',
                   lease_until = now()+5m   WHERE id=42 AND status='claimed'
                   -> succeeds. Worker-A now "owns" job 42 until t=300s.

t=10s    Worker-A starts executing job 42: "charge card for order #123"
                   -> begins the HTTP call to the payment provider

t=15s    Worker-A hits a 20-second GC pause (large heap, bad luck)
                   -- or: network blip stalls the HTTP call for minutes --
                   Worker-A is NOT crashed. It is just not responding.

t=300s   Lease sweep runs:
            UPDATE jobs SET status='pending'
              WHERE status='running' AND lease_until < now()
            -> job 42 qualifies (lease_until was t=300s, now it's stale)
            -> job 42 is back in the pool, claimable again

t=301s   Worker-B claims job 42 (status='pending' -> 'claimed' -> 'running')
                   -> starts executing: "charge card for order #123"
                   -> calls the payment provider -> CARD CHARGED (2nd time)

t=305s   Worker-A's GC pause ends. It has NO IDEA time has passed.
                   -> finishes its original HTTP call -> payment provider
                      confirms success (the FIRST charge)
                   -> Worker-A writes: UPDATE status='succeeded' WHERE
                      locked_by='A'   <- may or may not still match!

          RESULT: the card was charged twice. Two workers both believe
          they "own" job 42 at the same real-world instant.
```

The dangerous window is exactly `[lease start, lease sweep detects staleness]` overlapping with `[the side effect actually happening out in the real world]`. Nothing in this timeline is a bug — every individual step is "correct" given what that component could observe locally. The failure is a property of the *system*, not of any one line of code — which is exactly why retries/leases alone can't fix it.

Ranked fixes

1. Fencing tokens (closes the "zombie writes back" hole, but not the double side-effect). Give every lease a monotonically increasing token (`lease_epoch`). Worker-A's final `UPDATE ... WHERE locked_by='A' AND lease_epoch=7` fails once Worker-B has claimed with `lease_epoch=8` — the CAS in section 12 already does most of this. This stops Worker-A from *corrupting the record* after the fact (e.g. flipping a `dead_letter` row back to `succeeded`), but by t=301s the card is **already charged twice out in the real world** — a fencing token on the database write can't reach back and un-charge a card. It fixes the metadata race, not the side-effect race.

2. Idempotent job design (the practical answer, since true exactly-once is hard). Make the job's *effect* safe to apply more than once. Instead of "charge the card," the job does "charge the card **for idempotency key** `order-123-charge`," and the payment provider (Stripe and similar APIs support this natively) deduplicates on that key server-side — Worker-B's duplicate call returns the *same* charge result as Worker-A's, no second charge. For effects with no native idempotency key support, keep an internal dedup table: `INSERT INTO completed_effects (key) VALUES ('order-123-charge')` with a unique constraint, checked *before* performing the effect and written atomically as part of it. This is the one fix that actually closes the real-world double-execution gap, not just the bookkeeping gap — everything else on this list reduces *how often* the race happens; only this makes it *harmless* when it does.

3. Lease renewal / heartbeat. A long-running job periodically extends its own lease (`UPDATE ... SET lease_until = now()+5m WHERE locked_by='A'`) every, say, 60s while it's actively working. This shrinks the window in which a merely-slow-but-alive worker gets mistakenly declared dead — a heartbeat proves liveness continuously rather than betting on one upfront TTL. It does **not** help the GC-pause case in the timeline above: a paused process can't send heartbeats either, so a long enough pause still triggers the same race. Heartbeats reduce false-positive reclaims; they don't eliminate the fundamental gap.

4. Lease/visibility-timeout tuning. Set the TTL close to the job's real P99 runtime (not a generous round number like "5 minutes" for a 2-second job) so the reclaim window is as tight as correctness allows. Too short and you get *spurious* double-claims on jobs that are simply running a bit long; too long and a truly-dead worker's job sits idle for ages. This is a dial, not a fix — it changes the odds, not the outcome when the race does happen.

5. At-least-once + dedup at the effect boundary. The general form of fix #2: accept that delivery is at-least-once (section 9 already commits to this), and put a dedup check as close as possible to wherever the *external, non-idempotent* side effect happens — a payment provider's idempotency key, a `unique_violation` on an `INSERT`, a "has this email's Message-ID already been sent" check. The scheduler's job is to guarantee the job runs *at least* once and never zero times; the job's own logic is responsible for making a second run a no-op.

Recommended approach

Combine fencing tokens (cheap, closes the metadata-corruption hole, already almost-free given the CAS claim we already do) **with idempotent job design as the load-bearing fix** — that's the only one of the five that neutralizes the actual double-charge, not just the accounting around it. Tune the lease TTL to the job's real runtime and add heartbeats for anything long-running, purely to make the race *rarer*, not to claim it's gone.

> **What to say out loud:** "*I can't prevent a worker from pausing past its lease — GC, network stalls, and preemption are facts of life in any distributed system, and no amount of lease tuning eliminates the race, it only narrows it. So I don't try to make execution exactly-once; I make a second execution indistinguishable in effect from the first, via an idempotency key at the actual side-effect boundary — charge the card, send the email, whatever it is — backed by a fencing token so a zombie worker can't even corrupt the bookkeeping after the fact. That's a deliberately weaker mechanical guarantee that produces the same strong guarantee the business actually cares about: the customer is charged once.*"

Summary — the mental model

A distributed job scheduler is **not a scale-the-reads problem; it's "trigger the right work at the right time, hand it off durably, and survive every possible crash without ever silently losing a job."** The winning moves: an **indexed due-time query plus a time-wheel** so finding "what's due now" never means scanning millions of rows; an **atomic CAS claim** so two schedulers or workers can never run the same job at the same instant; a **durable queue** that decouples "decided it's due" from "actually executed"; **leases + backoff + a dead-letter queue** so a crashed worker or a poison job degrades gracefully instead of looping or vanishing; and, because true exactly-once is a mirage in a distributed

system, a deliberate commitment to **at-least-once delivery made safe by idempotency keys**. Everything else — priorities, cron, multi-region, GPUs — is a variation on routing jobs into the right queue for the right worker pool at the right time.